

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Спутниковая телеметрия: решение 10 вопросов
одним скриптом.

Запуск:
    python solve_space_risk.py [путь/
к/space_risk_model.csv]

Результат: submission.csv и
submission_space.csv (одинаковые файлы)
с колонками question_id, answer_1,
answer_2, answer_3.

Зависимости: numpy, pandas, scikit-learn.
"""
import csv
import io
import itertools
import sys
import time

import numpy as np
import pandas as pd
from sklearn.linear_model import
LinearRegression
from sklearn.metrics import roc_auc_score

# -----
```

```

-----
# Настройки
# -----
-----
DATA_PATH = sys.argv[1] if len(sys.argv) >
1 else "space_risk_model.csv"
OUT_PATHS = ["submission.csv",
"submission_space.csv"]
EXPECTED_ROWS = 180_300

# В вопросе 5 написано просто "battery".
Берём температуру батареи
# (тепловая связь с температурой солнечной
панели). Поменять можно здесь.
Q5_BATTERY_COL = "battery_temp_mean"
Q5_PRINT_ALTERNATIVES = True #
дополнительно напечатать результат для
других колонок батареи

MODELS = list(range(1, 11))
PRED_COLS = [f"model_{m}_pred" for m in
MODELS]

# -----
-----
# Загрузка данных
# -----
-----
def load_data(path):
    """Читает CSV. Если файл повреждён
(нулевые байты, строки с неверным числом
полей), битые строки отбрасываются, а в
консоль выводится предупреждение."""

```

```

raw = open(path, "rb").read()
df = None
if b"\x00" not in raw:
    try:
        df =
pd.read_csv(io.BytesIO(raw))
    except pd.errors.ParserError:
        df = None

if df is None:
    lines = raw.split(b"\n")
    header =
lines[0].strip().rstrip(b"\xef\xbb\xbf")
    ncol = header.count(b",") + 1
    good, bad = [], 0
    for ln in lines[1:]:
        ln = ln.strip()
        if not ln:
            continue
        if b"\x00" in ln or
ln.count(b",") + 1 != ncol:
            bad += 1
            continue
        good.append(ln)
    df = pd.read_csv(io.BytesIO(header
+ b"\n" + b"\n".join(good)))
    print(f"[ВНИМАНИЕ] файл повреждён:
отброшено битых строк: {bad}")

df = df.apply(pd.to_numeric,
errors="coerce").dropna()
for c in ("satellite_id",
"orbit_number", "orbit_type",
"failure_in_12h"):

```

```

        df[c] = df[c].astype(np.int64)
    df =
df.drop_duplicates(["satellite_id",
"orbit_number"])
    df = df.sort_values(["satellite_id",
"orbit_number"], kind="mergesort")
    df = df.reset_index(drop=True)

    print(f"Строк загружено: {len(df)}
(ожидалось {EXPECTED_ROWS}), "
        f"спутников:
{df.satellite_id.nunique()}, "
        f"доля отказов:
{df.failure_in_12h.mean():.4f}")
    if len(df) != EXPECTED_ROWS:
        print(f"[ВНИМАНИЕ] не хватает
{EXPECTED_ROWS - len(df)} строк "
            f"— ответы могут отличаться
от эталонных!")
    return df

# -----
# -----
# Вспомогательные функции
# -----
# -----

def pearson(x, y):
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    if x.size < 2:
        return np.nan
    xc, yc = x - x.mean(), y - y.mean()
    den = np.sqrt((xc * xc).sum() * (yc *

```

```

yc).sum())
    return float((xc * yc).sum() / den) if
den > 0 else np.nan

```

```

def value_at_lag(df, col, k):
    """Значение col на витке (orbit_number
- k) того же спутника, NaN если витка нет.
    Для непрерывной нумерации витков
совпадает с groupby().shift(k), но не
"склеивает" витки через пропуски."""
    s = df.set_index(["satellite_id",
"orbit_number"])[col]
    idx = pd.MultiIndex.from_arrays(
        [df["satellite_id"].to_numpy(),
(df["orbit_number"] - k).to_numpy()]
    )
    return s.reindex(idx).to_numpy()

```

```

# -----
-----
# Вопрос 1
# -----
-----
def q1(df):
    sat = df.groupby("satellite_id").agg(
        m=("solar_panel_temp_mean",
"mean"), ot=("orbit_type", "first")
    )
    return [int(sat.loc[sat.ot == t,
"m"].idxmax()) for t in (0, 1, 2)]

```

```

# -----
# -----
# Bonpoc 2
# -----
# -----

def q2(df):
    y = df["failure_in_12h"].to_numpy()
    aucs = {}
    for m in MODELS:
        ranks =
pd.Series(df[f"model_{m}_pred"].to_numpy())
.rank(method="first")
        aucs[m] = roc_auc_score(y,
ranks.to_numpy())
        top5 = sorted(MODELS, key=lambda m: (-
round(aucs[m], 12), m))[:5]
        return top5, [aucs[m] for m in top5],
aucs

# -----
# -----
# Bonpoc 3
# -----
# -----

def q3(df):
    solar = "solar_panel_temp_mean"
    d = df[["satellite_id", "orbit_number",
"failure_in_12h", solar]].copy()
    d["hr"] = d[solar].to_numpy() -
value_at_lag(d, solar, 3)
    d["hr_prev"] = value_at_lag(d, "hr",
3)          # heating_rate на витке k-3
    a_mask = (d.failure_in_12h == 0) &

```

```

d.hr.notna()
    b_mask = (d.failure_in_12h == 1) &
d.hr_prev.notna()
    A = d.loc[a_mask, "hr"].abs().mean()
    B = d.loc[b_mask,
"hr_prev"].abs().mean()
    return int(round(B / A)), A, B

# -----
# -----
# Bonpoc 4
# -----
# -----
def q4(df):
    y = df["failure_in_12h"].to_numpy() ==
1
    fpr = {}
    for m in MODELS:
        p =
df[f"model_{m}_pred"].to_numpy()
        t = p[y].min()
        alarm = p >= t
        fp = int((alarm & ~y).sum())
        tn = int((~alarm & ~y).sum())
        fpr[m] = fp / (fp + tn)
    best = min(MODELS, key=lambda m:
(fpr[m], m))
    return best, fpr[best], fpr

# -----
# -----
# Bonpoc 5

```

```

# -----
-----
def cross_corr_by_lag(df, battery_col,
lags=range(-5, 6)):
    """lag: corr(solar(t), battery(t+lag))
по каждому спутнику (только доступные
пары), затем простое среднее по
спутникам."""
    x_all =
df["solar_panel_temp_mean"].to_numpy()
    sat_all = df["satellite_id"].to_numpy()
    mean_r = {}
    for lag in lags:
        b = value_at_lag(df, battery_col, -
lag) # battery на витке t+lag
        ok = ~np.isnan(b)
        tmp = pd.DataFrame({"s":
sat_all[ok], "x": x_all[ok], "y": b[ok]})
        rs = [pearson(g["x"], g["y"]) for
_, g in tmp.groupby("s")]
        mean_r[lag] = float(np.nanmean(rs))
    return mean_r

def q5(df):
    mean_r = cross_corr_by_lag(df,
Q5_BATTERY_COL)
    best_lag = max(mean_r, key=lambda k:
(mean_r[k], -k))
    if Q5_PRINT_ALTERNATIVES:
        for col in ("battery_temp_mean",
"battery_voltage_mean",
"battery_current_mean"):
            if col == Q5_BATTERY_COL:

```



```

        continue
        alt = cross_corr_by_lag(df,
col)
        k = max(alt, key=lambda z:
(alt[z], -z))
        print(f"    (для справки:
{col}: лучший lag={k}, r={alt[k]:.3f})")
        return best_lag, mean_r[best_lag],
mean_r

# -----
# -----
# Вопрос 6
# -----
# -----

def partial_corr(x, y, z):
    r = np.corrcoef(np.vstack([x, y, z]))
    rxy, rxz, ryz = r[0, 1], r[0, 2], r[1,
2]
    return (rxy - rxz * ryz) / np.sqrt((1 -
rxz ** 2) * (1 - ryz ** 2))

def q6(df):
    s =
df["solar_panel_temp_mean"].to_numpy()
    i =
df["battery_current_mean"].to_numpy()
    v =
df["battery_voltage_mean"].to_numpy()
    r1 = partial_corr(s, i, v)    #
X=solar, Y=current, Z=voltage
    r2 = partial_corr(i, v, s)    #

```

```

X=current, Y=voltage, Z=solar
    return max(abs(r1), abs(r2)), r1, r2

# -----
# -----
# Вопрос 7
# -----
# -----

def max_f1_over_thresholds(score, y,
n_pos):
    """Максимум F1 по всем уникальным
    порогам t (опасен, если score > t)."""
    order = np.argsort(-
score)                # порядок внутри
равных значений не важен
    s = score[order]
    tp = np.cumsum(y[order],
dtype=np.int64)
    bd = np.flatnonzero(s[:-1] >
s[1:])                # граница между разными
значениями
    if bd.size == 0:
        return 0.0
    k = bd +
1                        # число
сработавших витков
    f1 = 2.0 * tp[bd] / (k +
n_pos)                # 2TP/(2TP+FP+FN) =
2TP/(k+P)
    return float(f1.max())

def q7(df):

```

```

        a = (df["battery_temp_mean"] /
40.0).to_numpy()
        b = (df["reaction_wheel_current_mean"]
/ 2.0).to_numpy()
        c = (df["attitude_error_mean"] /
1.5).to_numpy()
        y =
df["failure_in_12h"].to_numpy().astype(np.i
nt64)
        n_pos = int(y.sum())
        best, best_w = -1.0, None
        for i in range(101):
            for j in range(101 - i):
                w1, w2, w3 = i / 100, j / 100,
(100 - i - j) / 100
                f1 = max_f1_over_thresholds(w1
* a + w2 * b + w3 * c, y, n_pos)
                if f1 > best:
                    best, best_w = f1, (w1, w2,
w3)
        return best, best_w

# -----
# -----
# Bonpoc 8
# -----
# -----

def q8(df):
    col = "reaction_wheel_current_std"
    g = df.groupby("satellite_id")[col]
    thr = (g.mean() + 3 *
g.std(ddof=0)).reindex(df["satellite_id"]).
to_numpy()

```

```

        anom = (df[col].to_numpy() >
thr).astype(float)
        frac =
pd.Series(anom).groupby(df["satellite_id"].
to_numpy()).mean()
        fails = df.groupby("satellite_id")
["failure_in_12h"].sum().reindex(frac.index
)
        return pearson(frac.to_numpy(),
fails.to_numpy())

# -----
-----
# Bonpoc 9
# -----
-----
def q9(df):
    X = df[["attitude_error_mean",
"reaction_wheel_current_mean"]].to_numpy()
    y = df["battery_temp_mean"].to_numpy()
    res = y - LinearRegression().fit(X,
y).predict(X)
    f = df["failure_in_12h"].to_numpy()
    return float(res[f == 1].std(ddof=1) /
res[f == 0].std(ddof=1))

# -----
-----
# Bonpoc 10
# -----
-----
class Q10Ctx:

```

```

    """Предрасчитанные константы для оценки
    ансамблей."""

    def __init__(self, df):
        self.cols = {m:
np.ascontiguousarray(df[f"model_{m}_pred"].
to_numpy()) for m in MODELS}
        self.y =
df["failure_in_12h"].to_numpy().astype(np.i
nt64)
        self.g =
df["orbit_type"].to_numpy().astype(np.int64
)
        self.N = len(df)
        self.M = self.N //
4 # (TP+FP)/N <= 0.25
<=> k <= N//4
        self.P = int(self.y.sum())
        self.Pg = [int(self.y[self.g ==
i].sum()) for i in range(3)]
        self.Ng = [int((self.g == i).sum())
for i in range(3)]

def ensemble_best(P, ctx):
    """Лучший допустимый порог для ансамбля
    P (опасен, если P > t).
    Возвращает (L, t) или None. Все
    ограничения проверяются в целых числах:
        глобально: FN/Pos <= 0.10,
alarms/N <= 0.25
        по orbit_type: FN_g/Pos_g <= 0.15,
alarms_g/N_g <= 0.30
    """

```

```

M = ctx.M
# Допустимы только пороги, при которых
сработало не больше M витков -> достаточно
топ-(M+1)
part = np.argpartition(-P, M)[: M + 1]
top = part[np.argsort(-
P[part])] # порядок внутри
равных значений не важен
s = P[top]
ys = ctx.y[top[:M]]
gs = ctx.g[top[:M]]
bd = np.flatnonzero(s[:M] > s[1: M +
1]) # границы между разными значениями
if bd.size == 0:
    return None
tp = np.cumsum(ys)[bd]
ok = 10 * (ctx.P - tp) <= ctx.P
if not ok.any():
    return None
for gi in range(3):
    mg = gs == gi
    a_g = np.cumsum(mg)[bd]
    tp_g = np.cumsum(mg & (ys == 1))
[bd]
    ok &= (20 * (ctx.Pg[gi] - tp_g) <=
3 * ctx.Pg[gi]) & (10 * a_g <= 3 *
ctx.Ng[gi])
    if not ok.any():
        return None
k = bd + 1
fn = ctx.P - tp
fp = k - tp
loss = 20_000_000 * fn + 100_000 * fp
loss = np.where(ok, loss,

```

```

np.iinfo(np.int64).max)
    lmin = loss.min()
    cand = np.flatnonzero(loss == lmin)
    t_vals = s[bd +
1]                                # порог =
ближайшее меньшее значение
    j =
cand[np.argmin(t_vals[cand])]
# при равных L - минимальный t
    return int(lmin), float(t_vals[j])

def compositions(total):
    """Все (i, j, k), i+j+k = total, i, j,
k >= 1 (веса > 0)."""
    for i in range(1, total - 1):
        for j in range(1, total - i):
            yield i, j, total - i - j

def search_weights(ctx, triples, total,
label=""):
    """Перебор весов с шагом 1/total.
Критерий: (L, t, тройка, веса) -
лексикографический минимум."""
    best = None
    for n_tr, tr in enumerate(triples, 1):
        pa, pb, pc = (ctx.cols[m] for m in
tr)
        for i, j, k in compositions(total):
            P = (i / total) * pa + (j /
total) * pb + (k / total) * pc
            r = ensemble_best(P, ctx)
            if r is None:

```

```

        continue
        cand = (r[0], r[1], tr, (i, j,
k))

        if best is None or cand < best:
            best = cand
            if label and (n_tr % 20 == 0 or
n_tr == len(triples)):
                print(f"    {label}:
{n_tr}/{len(triples)} троек, лучшее L
сейчас: "
                    f"{best[0] if best else
None}")
            return best

def q10(df):
    ctx = Q10Ctx(df)
    triples =
list(itertools.combinations(MODELS, 3))
    coarse = search_weights(ctx, triples,
20, "грубый поиск, шаг 0.05")
    if coarse is None:
        return None
    fine = search_weights(ctx, [coarse[2]],
100, "уточнение, шаг 0.01")
    best = min(coarse, fine) if fine is not
None else coarse
    L, t, tr, wi = best
    print(f"    грубо: L={coarse[0]},
тройка={coarse[2]}, веса={tuple(v / 20 for
v in coarse[3])}")
    print(f"    точно: L={best[0]}, тройка=
{tr}, веса={tuple(v / 100 for v in wi)}, t=
{t:.6f}")

```



```

        return tr, t, L

# -----
# -----
# main
# -----
# -----

def main():
    t0 = time.time()
    df = load_data(DATA_PATH)
    rows = []

    def stage(name):
        print(f"\n{name} ...", flush=True)

    stage("Bonpoc 1")
    ids = q1(df)
    print("    LEO/MEO/GEO:", ids)
    rows.append([1, ids[0], ids[1],
ids[2]])

    stage("Bonpoc 2")
    top5, auc5, _ = q2(df)
    print("    топ-5:", top5, [round(a, 4)
for a in auc5])
    rows.append([2, ",".join(map(str,
top5)), ",".join(f"{a:.2f}" for a in auc5),
""])

    stage("Bonpoc 3")
    ans3, A, B = q3(df)
    print(f"    A={A:.5f}  B={B:.5f}  B/A=
{B / A:.3f}")

```

```

rows.append([3, ans3, "", ""])

stage("Вопрос 4")
m4, f4, _ = q4(df)
print(f"    модель {m4}, FPR={f4:.4f}")
rows.append([4, m4, f"{f4:.2f}", ""])

stage("Вопрос 5")
lag, r5, _ = q5(df)
print(f"    lag={lag}, r={r5:.4f}")
rows.append([5, lag, f"{r5:.2f}", ""])

stage("Вопрос 6")
r6, r1, r2 = q6(df)
print(f"    r1={r1:.4f}, r2={r2:.4f}")
rows.append([6, f"{r6:.2f}", "", ""])

stage("Вопрос 7 (5151 набор весов)")
f7, w7 = q7(df)
print(f"    F1={f7:.5f} при весах
{w7}")
rows.append([7, f"{f7:.3f}", "", ""])

stage("Вопрос 8")
r8 = q8(df)
print(f"    r={r8:.5f}")
rows.append([8, f"{r8:.3f}", "", ""])

stage("Вопрос 9")
r9 = q9(df)
print(f"    std1/std0={r9:.4f}")
rows.append([9, f"{r9:.2f}", "", ""])

stage("Вопрос 10 (может занять

```

```

несколько минут)")
    res10 = q10(df)
    if res10 is None:
        print("    допустимых решений нет")
        rows.append([10, "", "", ""])
    else:
        tr, t, L = res10
        rows.append([10, " ".join(map(str,
tr)), f"{t:.2f}", int(L)])

        print("\n" + "=" * 60)

print("question_id,answer_1,answer_2,answer
_3")
    for r in rows:
        print(" ".join(f'"{c}"' if "," in
str(c) else str(c) for c in r))

        for path in OUT_PATHS:
            with open(path, "w", newline="",
encoding="utf-8") as f:
                w = csv.writer(f,
lineterminator="\n")
                w.writerow(["question_id",
"answer_1", "answer_2", "answer_3"])
                w.writerows(rows)
            print(f"\nСохранено: {'
'.join(OUT_PATHS)}    (время: {time.time() -
t0:.0f} c)")

if __name__ == "__main__":
    main()

```

